Group Assignment 2:
In the name of deep learning

Computer Vision (H02A5a)

In this group assignment your team will delve into some deep learning applications for computer vision. The assignment will be delivered in the same groups from *Group assignment 1* and you start from the provided *Starter Code* on Toledo (Content > Assignments > Group Assignment 1 > Starter Code). You can make use of the *Group assignment 2* forum/discussion board on Toledo if you have any questions. Good luck and have fun!

1 Overview

This assignment consists of *three main parts* for which we expect you to provide code and documentation in the notebook:

- Image classification (Sect. 2.1)
- Semantic segmentation (Sect. 2.2)
- Adversarial attacks (Sect. 2.3)

In the first part, you will train an end-to-end neural network for image classification. In the second part, you will do the same for semantic segmentation. For these two tasks we expect you to put a significant effort into optimizing performance and as such competing with fellow students via the Kaggle competition. In the third part, you will try to find and exploit the weaknesses of your classification and/or segmentation network. For the latter there is no competition format, but we do expect you to put significant effort in achieving good performance on the self-posed goal for that part. Finally, we ask you to reflect and produce an overall discussion with links to the lectures and "real world" computer vision (Sect. 2.4). It is important to note that only a small part of the grade will reflect the actual performance of your networks. However, we do expect all things to work! In general, we will evaluate the correctness of your approach and most importantly your insight into why you have used certain methods and why they are behaving in a certain way.

1.1 Deep learning resources

If you did not yet explore this in Group assignment 1 (Sect. 2), we recommend using Pytorch or TensorFlow/Keras for building deep learning models. The deep learning paradigm is

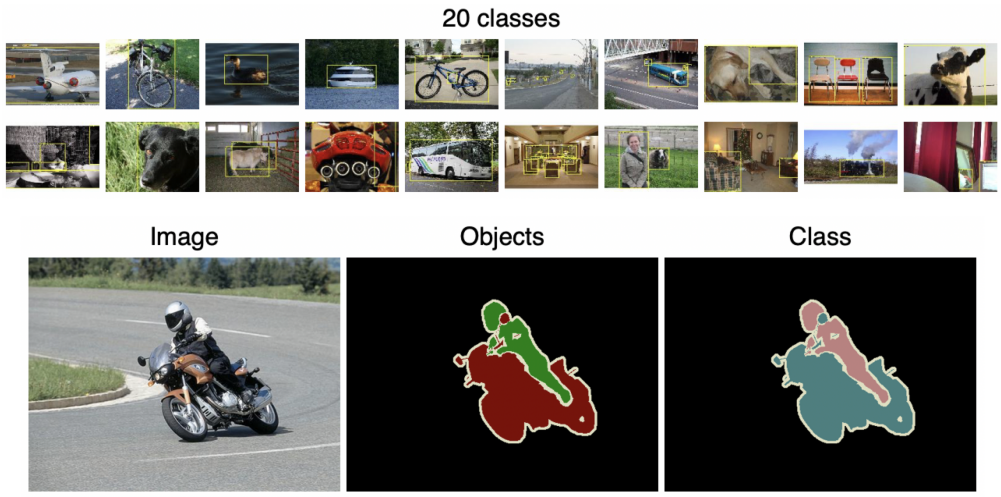

Figure 1: PASCAL VOC 2009 dataset [Eve+10] (source).

extensive but most of it is out of the scope for this course. Therefore, it is not necessary to explore different optimizers (e.g. SGD, ADAM, etc.), regularization mechanisms (e.g. L_p norm), varying non-linearities (e.g. ReLU, sigmoid), architectures (e.g. U-Net, DeepMedic, ResNet), etc. The main goal of this assignment is to gain a fundamental understanding of how deep learning can be used in computer vision applications and not to explore all the intricacies of deep learning itself. However, keep in mind that these things can lead to performance gains and as such may help you in the Kaggle competition! With a validated Kaggle account you can make use of GPU accelerated computing as well.

1.2 PASCAL VOC 2009

For this project you will be using the PASCAL VOC 2009 dataset [Eve+10]. This dataset consists of colour images of various scenes with different object classes (e.g. animal: *bird, cat, ...*; vehicle: *aeroplane, bicycle, ...*), totalling 20 classes (Fig. 1). The dataset has been used as a benchmark for classification, detection and segmentation challenges and for comparing results of individual research (more details can be found here). The specific subset of this dataset that will be used during this assignment is made available in the Kaggle competition. Ground truth image labels and segmentation masks are only available for the training set. It should be clear that multiple objects can be present in a single image, but that a single pixel can only belong to one object class. The classification task (Sect. 2.1) could therefore be interpreted as for each image: "What object classes are present in this image?" The segmentation task (Sect. 2.2) could be interpreted as for each image and for each pixel: "To what object class does this pixel belong?".

2 Assignment

2.1 Image classification

The goal here is simple: implement a classification network and train it to recognise all 20 classes (and/or background) using the training set and compete on the test set.

You can choose your own architecture, but we advise you to stick to a network architecture that is known to work. We encourage you to experiment with two types of models: (i) a model where you train all the parameters from scratch; and (ii) a model where you transfer weights from a different model and apply fine-tuning (you can use a pre-trained network from the python library you use or create one yourself). When you choose to do so, make sure that none of these pre-trained models have been trained using images from a PASCAL VOC dataset!

Think about the task that you are trying to solve and the dataset at your disposal, how does this influence the model and the loss function? Elaborate on the design choices of your data pipeline, network architecture, learning parameters and loss function. A few guiding questions could be:

- What layers do you use? What is the use of each layer?
- What kind of loss function do you use?
- Is your model learning as it should?
- How does it perform? Which mistakes does it make and why?

Lastly, put some effort in the optimization of your classification model so that you obtain competitive performance in the competition!

2.2 Semantic segmentation

The goal here is to implement a segmentation model that labels every pixel in the image as belonging to one of the 20 classes (and/or background). Use the training set to train your network and compete on the test set.

You can design the architecture as you see fit but we recommend again to start from known segmentation architectures that have been reported in literature. Again, elaborate on the design choices of your network architecture, learning parameters and loss function. A few additional guiding questions for segmentation could be:

- What is the difference with a classification setup?
- What kind of loss function do you use?
- Is your model learning as it should?
- How does it perform? Which mistakes does it make and why?

Lastly, put some effort in the optimization of your segmentation model so that you obtain competitive performance in the competition!

2.3 Adversarial attacks

As you will undoubtedly have noticed in the previous exercises, computer vision has seen substantial performance improvements in the last decade, even outperforming humans in certain tasks. However, computer vision is still far away from being *a solved problem*. One important limitation is that the state-of-the-art today, unlike humans, does not have prior knowledge about its environment, it can only learn from what is present in the training data. By using a lot of data in the training phase, the model is assumed to be able to generalise outside of the training set, and we attempt to validate this using an independent test set. However, it is still very likely that our dataset is biased in one way or another, so even though your model might show good performance in the lab environment it might fail when being released "in the wild". Look up "Tesla autopilot fails" on YouTube to see some examples. The seemingly unavoidable problem of bias has resulted in some skeptics saying that machine learning technologies (like deep learning) are not ready to be adopted in our daily lives. We (the TA's) do not follow this line of thinking but we do encourage you (the future AI experts) to evaluate what you read, and develop with a critical mindset and to be aware of the limitations. The interested student is referred to this nice tutorial session by J.Z. Kolter and A. Madry.

In an adversarial attack, an adversary (attacker) adds noise (perturbation) to an image in an attempt to fool the system under attack. Figure 2 is a famous example of such adversarial input that was generated using the fast gradient sign method, where the initial model was trained to recognizes animals, but it classifies and unnoticeably perturbed panda image as a gibbon. The adversary could be a model that was trained/configured by a hacker or it might simply be a simulation of the real world input variability that you use to gain a better understanding of the limitations of your model or to improve robustness. The system under attack could be a cloud API, a spam filter or an automated vehicle. We distinguish two levels of knowledge/access that the adversary can have over the system under attack: white-box and black-box. In the case of a white-box attack, the attacker has full knowledge and access to the model under attack and its weights (e.g. open source implementation, your own implementation). In a black-box attack, the adversary only has knowledge of the input of the model under attack and the related output (e.g. an API, automated vehicle with encrypted source code).

For this part, your goal is to fool your classification and/or segmentation model, using an *adversarial attack*. More specifically, the goal is to build a network to perturb test images in a way that (i) they look unperturbed to humans; but (ii) the model classifies/segments these images in line with the perturbations. In this case, you can build an encoder-decoder type network that learns how to perturb panda images such that you can fool/attack your initial classifier.

An overview of this type of adversarial model is shown in Figure 3. Here, the adversarial model $f_{adv}(x;\theta_{adv})$ takes in the original input x and generates a perturbation $\delta = f_{adv}(x)$, which is then added to the original input before it is passed to your unchanged classification model $h_{cls}(x;\theta_{cls})$. The objective of your adversary is to generate the perturbation in such a way that the classifier h_{cls} assigns a class/label map to the perturbed image $\hat{y}$ that is different from the ground truth y (it can be a specifically chosen y_{adv} in what is called a

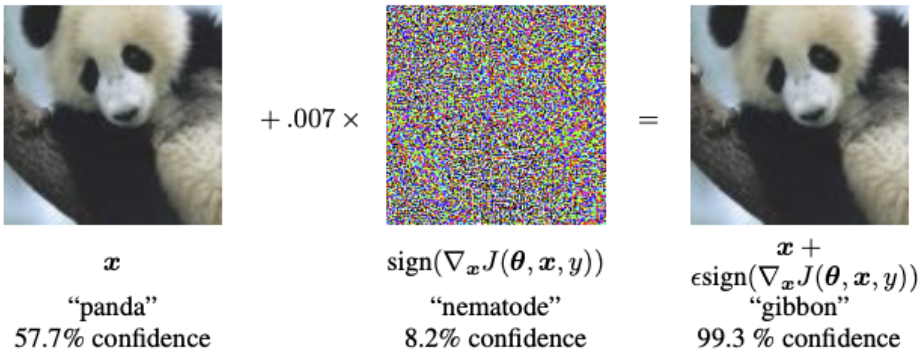

Figure 2: Adversarial example (from [GSS15]).

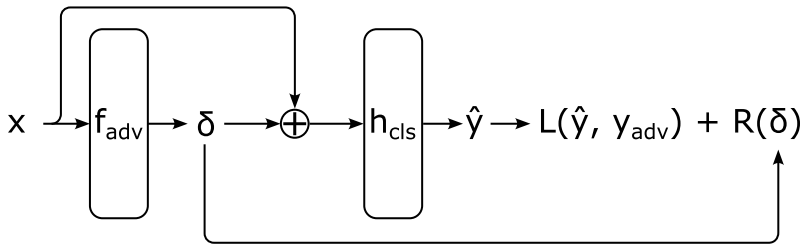

Figure 3: Adversarial model diagram.

targeted attack, or simply any wrong label in an untargeted attack). If you look back at the goal for this part in the previous paragraph, it should be clear that the regularization term R and the loss function L will take care of (i) and (ii), respectively. It is very important that you only train the adversary's parameters θ_{adv} while the original model's parameters θ_{cls} stay frozen (e.g. unchanged).

In this assignment we advise you to focus on the white-box attack, which is generally considered to be the simplest since you have full access to the full model under attack (and thus its gradients!).

While you can choose your set-up, clearly explain your methods and choices:

- Define your task: e.g. we are going to perturb our images such that our classification model says there are aeroplanes in it, while there are not; we are going to perturb our images such that our segmentation model will segment chairs in it, while there aren't any.
- Which losses do you use for the task and the regularisation?
- Give some visual examples of what you are doing, e.g. the test image, the perturbation, the perturbed test image.
- Reflect on your setup/approach: Is this a realistic attack? Can it be used in a "real world" scenario? If so, how/when? Was the adversary successful? Are the images still recognisable for a human observer? Does this mean that your initial model is now

unreliable? What could we do to increase the robustness of a model under attack in general?

2.4 Discussion

Finally, take some time to reflect on what you have learned during this assignment. Reflect and produce an overall discussion with links to the lectures and "real world" computer vision. We don't expect that you do a full literature study on these tasks but think critically about what you did and the results that you achieved. Did your classification and segmentation networks perform well? Has your adversary reached its goal? What would you do differently if you had some more time? Where would you spend your time on to improve your results? What are the limitations of your models? Are they ready to be deployed in a "real world" setting? Feel free to add any other critical reflections you might think of. The most important part of the assignment is to learn to interpret your models, only then you can improve them or develop new ones.

3 Practicalities

3.1 Kaggle: Joining the competition

Closely follow the instructions on Toledo for joining the competition, sharing your notebook with the TAs and making a valid notebook submission to the competition.

In order to compete in the Kaggle competition each team should come up with image labels and segmentation maps for the test set during Sect. 2.1 and Sect. 2.2. However, to fit Kaggle's internal scoring mechanism, a valid submission should output a CSV file for which a single metric is computed for each row. Therefore, we have provided a *submit_df_to_competition* function in the template notebook, that will convert your predictions to a valid *submission.csv* file. This file now has two rows per example (one for classification and one for segmentation) and the predictions for all the classes are now grouped into a single column. Both for classification and segmentation the predictions are running length encoded and for each we compute the Dice score. Optionally, you can have a look at this function in more detail. Just make sure to call this function once after you have completed Sect. 2.1 and Sect. 2.2.

3.2 Submission

The notebook you submit for grading is the last notebook you submit in the Kaggle competition‘ prior to the deadline (you can submit as many times as you want, we strongly recommend that you do so regularly). Only submissions through Kaggle will be accepted (time and memory issues are your own responsibility). Any submissions after the deadline will be disregarded.

A notebook submission not only produces a *submission.csv* file that is used to calculate your competition score, it also runs the entire notebook and saves its output as if it were a report. This way it becomes an all-in-one-place document for the TAs to review. As such,

please make sure that your final submission notebook is self-contained and fully documented (e.g. provide strong arguments for the design choices that you make). Most likely, this notebook format is not appropriate to run all your experiments at submission time (e.g. the training is a memory hungry and time consuming process) due to limited Kaggle resources. It can be a good idea to distribute your code otherwise and only summarize your findings, together with your final predictions, in the submission notebook. For example, you can substitute experiments with some text and figures that you have produced "offline". We advise you to go through this document entirely before you really start. It can be a good idea to then go through the template notebook and use this one as your first notebook submission to the competition.

To add your TAs to your Kaggle team, look for their Kaggle usernames on Toledo under *Didactic Team*.

References

[Eve+10] Mark Everingham et al. "The pascal visual object classes (voc) challenge". In: *International journal of computer vision* 88.2 (2010), pp. 303–338.

[GSS15] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. "Explaining and Harnessing Adversarial Examples". In: *arXiv:1412.6572 [cs, stat]* (Mar. 2015). arXiv: 1412.6572. URL: http://arxiv.org/abs/1412.6572 (visited on 03/12/2020).